High Level Design

NeoBank

Project Digital Leap

Revision	Date	Author	Details
0.1	07/03/2026	Luca Agrippino	First draft: requirements, assumptions, constraints
0.2	07/03/2026	Luca Agrippino	Section 3 first draft: component list, design principles, system flows
0.3	10/03/2026	Luca Agrippino	Use Case Diagrams (UCD-01/02/03), Deployment Diagram, actor analysis
0.4	15/03/2026	Luca Agrippino	Domain Model, State Machine, Activity Diagram, all Sequence Diagrams, Component Diagrams, Lambda Architecture
0.5	19/03/2026	Luca Agrippino	Corrected component placement, architecture patterns, technology decisions, security analysis, message schemas, sizing, upgradability, time estimation

Contents

Contents 2
Abstract 4
About This Document 5
1. General 6
1.1 Introduction 6
1.2 Glossary 7
2. Requirements 8
2.1 Assumptions 8
2.2 Functional Requirements 9
2.3 Non-Functional Requirements 10
2.3.1 Availability and Recovery 10
2.3.2 Performance and Capacity 10
2.3.3 Security 10
2.3.4 Backward Compatibility 10
2.3.5 Upgradability 11
2.3.6 Monitoring and Observability 11
2.3.7 Deployment 11
2.3.8 Compliance 11
2.3.9 Cost and Operational Efficiency 11
2.4 Requirement Tensions and Trade-offs 12
3. High Level Design 13
3.0 Use Case Analysis 13
3.0.1 Actors 13
3.0.2 Use Cases 14
3.0.3 Use Case Diagrams 15
3.1 Deployment and Component Architecture 17
3.1.1 Component Inventory 18
3.1.2 Component Diagrams 19
3.1.3 Domain Model 21
3.2 Technology Decisions 22
3.3 Design Principles and Architecture Patterns 23
3.3.1 Design Principles 23
3.3.2 Structural Patterns 23
3.3.3 Data Patterns 24
3.3.4 Resilience Patterns 25
3.3.5 Messaging Patterns 26
3.3.6 Deployment Patterns 27
3.4 Security Architecture 28

3.5 High Level System Flows 29
3.5.1 Transaction Lifecycle 29
3.5.2 Account Balance Query (Synchronous Read) 30
3.5.3 Fund Transfer with Fraud Detection (Synchronous Write) 31
3.5.4 Login and Token Issuance 33
3.5.5 AI Financial Advice 34
3.5.6 GDPR Data Deletion 35
3.5.7 Open Banking Payment 36
3.5.8 CDC Data Pipeline 37
3.6 Message Schemas 38
3.7 Upgradability 39
3.8 Sizing 40
4. Time Estimation 41
5. Limitations 42
6. Risks and Mitigations 43
7. Open Issues 44

Abstract

This document describes the high-level architecture for NeoBank's Project Digital Leap. The project is a strategic initiative to modernise the bank's digital services while preserving the stability and regulatory compliance of the existing IBM iSeries Mainframe infrastructure.

We propose a hybrid deployment model. A new on-premises layer handles all real-time transactional processing alongside the mainframe. Cloud-hosted services handle analytical workloads, AI-based advisory, and the public-facing API endpoints. The two environments are connected through a private encrypted tunnel, and no customer data ever crosses the public internet unencrypted.

The system targets 100,000 users at launch, scaling to 1,000,000 within three years. It guarantees 99.999% availability, zero data loss, and full GDPR compliance. The Mainframe remains the authoritative system of record throughout the MVP phase. We build a bridge to it, not a replacement for it.

About This Document

The purpose of this document is to give all stakeholders a shared understanding of the system's design before detailed design and implementation work begins. It covers the system architecture, component structure, data flows, deployment topology, and all external dependencies.

This is a living document and will be updated as design decisions are refined and validated.

Intended audience:

Stakeholders and Product Management: to verify that all business requirements are addressed and that the architecture supports NeoBank's strategic goals.

Architects and Lead Engineers: to understand component boundaries, data flows, technology choices, and the reasoning behind them.

Operations and Infrastructure Teams: to understand what needs to be provisioned, monitored, and maintained.

1. General

1.1 Introduction

NeoBank is not starting from a blank page. This is a modernisation of a live, mission-critical banking system that processes real money for real customers every day. The IBM iSeries Mainframe has been running reliably for decades, and it is not going away. Every decision in this document respects that reality.

The goal of Project Digital Leap is to build a modern digital services layer that makes the value inside the legacy system accessible to customers through a mobile app, a web portal, and Open Banking APIs. We are building a bridge between the legacy mainframe world and the modern cloud-native world. The bridge must be reliable enough that neither side notices the other's limitations.

On the functional side, the new platform will let customers check their balance, transfer funds, receive monthly reports, and get AI-powered financial advice. It will also open the bank to third-party fintech services through standardised Open Banking APIs, as required by PSD2 regulations.

On the non-functional side, the numbers are demanding: 99.999% uptime (about five minutes of downtime per year), zero data loss under any failure, millisecond response times, and full GDPR compliance including the right to be forgotten. These requirements, combined with the constraint that all transactions must pass through the legacy CBS, create tensions that drive most of the architectural decisions in this document.

1.2 Glossary

Term	Definition
CBS	Core Banking System, runs on the Mainframe and processes all financial transactions.
Mainframe	IBM iSeries server running COBOL on z/OS. The system of record for all financial data.
DB2	Relational database on the Mainframe that stores all transaction records.
ADABAS	Database on the Mainframe that stores customer profile and personal information.
CDC	Change Data Capture, a pattern for streaming data changes from a source system in near-real-time.
HLD	High-Level Design, this document.
MVP	Minimum Viable Product, the first production release targeted within 12 months.
GDPR	General Data Protection Regulation, the EU data privacy law.
PSD2	Payment Services Directive 2, the EU regulation that mandates Open Banking APIs.
API Gateway	Cloud-hosted entry point for all external client traffic.
Partner Gateway	Cloud-hosted entry point specifically for third-party fintech providers.
IdP	Identity Provider, handles authentication and authorisation (implemented with Keycloak).
BFF	Backend for Frontend, the on-premises service that processes all client requests.
Integration Layer	Bridges the Mainframe and the modern data layer in both directions.
SoR	System of Record, the authoritative source for a given piece of data.
RPO	Recovery Point Objective, the maximum acceptable data loss measured in time. RPO = 0 means none.
RTO	Recovery Time Objective, the maximum acceptable downtime before a service must be restored.
TPS	Transactions Per Second.
PII	Personally Identifiable Information, any data subject to GDPR.

2. Requirements

This section translates NeoBank's business objectives into concrete, testable technical requirements. We start with assumptions and constraints, which define the boundaries. Then we define what the system must do (functional requirements) and how well it must do it (non-functional requirements). The section closes with an explicit list of tensions between requirements. These tensions are the primary drivers of the design decisions in Section 3.

2.1 Assumptions

These must be true for the architecture to be valid. If any assumption proves false, the parts of the design that depend on it will need to be revisited.

ID	Assumption	Risk if False
A-01	We can extract data from DB2 and ADABAS through CDC or a mainframe API gateway, without rewriting the COBOL codebase.	Critical. The entire integration strategy is invalid. Must validate in the first two weeks.
A-02	AWS infrastructure and CI/CD pipelines will be ready at project kickoff. The team can start building from day one.	Ramp-up delays directly threaten the 12-month deadline.
A-03	Customer data can be sufficiently anonymised before leaving the on-premises boundary, while still being useful for AI financial advice.	The AI Agent feature must be descoped from the MVP.
A-04	The Mainframe will remain operational for the full MVP duration. No planned decommissioning or major hardware changes.	The entire architecture becomes invalid.
A-05	The 60-developer team can work in parallel streams without blocking each other.	Sequential delivery would make the 12-month timeline impossible.

2.2 Functional Requirements

Each requirement uses the word 'shall' and is phrased to be testable. It describes an observable behaviour, not an intention.

ID	Requirement
FR-01	The system shall provide a mobile and web interface that allows authenticated users to view their current account balance and full transaction history.
FR-02	The system shall allow authenticated users to initiate a fund transfer to any account within NeoBank or to an external bank account.
FR-02.1	Every fund transfer shall be analysed by the Fraud Detection Engine in real-time before submission to the CBS. Fraudulent transactions shall be blocked.
FR-02.2	The Fraud Detection Engine shall also operate in an offline mode, processing historical data to detect complex patterns that real-time checks cannot catch.
FR-02.3	When a transaction is cancelled due to fraud, the system shall immediately notify the user via the mobile or web application.
FR-03	The system shall generate a monthly income and expense report for each user.
FR-04	The system shall expose RESTful Open Banking APIs for authorised third-party fintech providers, subject to explicit and revocable user consent.
FR-05	The system shall provide an AI Financial Agent offering personalised advice based on the user’s transaction history.
FR-05.1	The AI Agent shall operate exclusively in the cloud and shall only receive anonymised or aggregated data in compliance with GDPR.

2.3 Non-Functional Requirements

These requirements shape the architecture more than the functional ones. They determine what technologies we use, how we deploy, and how much everything costs.

2.3.1 Availability and Recovery

ID	Requirement
NFR-01	99.999% availability for all customer-facing services (approximately 5 minutes and 15 seconds of unplanned downtime per year).
NFR-02	Zero data loss (RPO = 0) under any single component failure.
NFR-03	RTO for critical services to be defined after disaster recovery analysis.

2.3.2 Performance and Capacity

ID	Requirement
NFR-04	Support 100,000 digital users at MVP launch without degradation.
NFR-05	Scale to 1,000,000 users within 3 years, same geographic region.
NFR-06	Millisecond-level latency for all customer-facing API responses.
NFR-07	Support thousands of concurrent users while maintaining latency targets.

2.3.3 Security

ID	Requirement
NFR-08	Every user must be authenticated via a dedicated identity provider before accessing any service.
NFR-09	Authorisation controls must ensure that each user and third-party app can only access data and operations they have been explicitly granted.
NFR-10	All data at rest and in transit must be encrypted. No plaintext PII anywhere.
NFR-11	The digital services layer must be architecturally segregated from the Mainframe. No direct internet-to-Mainframe path is permitted.

2.3.4 Backward Compatibility

ID	Requirement
NFR-12	Data consistency with legacy DB2 and ADABAS throughout MVP. The Mainframe remains the system of record.
NFR-13	App-initiated transactions must be reflected immediately. External transactions may be up to 24 hours stale.
NFR-14	All financial transactions must be CBS transactions. Nothing is final until the Mainframe confirms it.

2.3.5 Upgradability

ID	Requirement
NFR-15	Deploy new versions without full downtime. No data loss or broken in-flight transactions during upgrades.
NFR-16	Schema migrations require explicit, documented plans. No backward-incompatible changes without coordination.
NFR-17	Maintenance windows to be agreed with business in a subsequent revision.

2.3.6 Monitoring and Observability

ID	Requirement
NFR-18	Centralised logging, metrics, and distributed tracing across all components in both environments.
NFR-19	Dashboards for technical metrics (uptime, error rates, p50/p95/p99 latency, TPS) and business metrics (transaction volumes, active users, fraud rates).
NFR-20	All components must expose health-check endpoints. Automated alerting within 60 seconds of degradation.

2.3.7 Deployment

ID	Requirement
NFR-21	Core transactional services and legally restricted services must run on-premises, isolated from the public internet.
NFR-22	The AI Agent, API Gateway, and Partner Gateway must be in a public cloud within the same geographic region as customers.
NFR-23	All PII must remain within the geographic region. No cross-region transfer.

2.3.8 Compliance

ID	Requirement
NFR-24	Full GDPR compliance, including Right to be Forgotten. Personal data must be deletable across all layers.

2.3.9 Cost and Operational Efficiency

ID	Requirement
NFR-25	Minimise Mainframe utilisation. Offload reads to the modern layer and reserve the Mainframe for writes only.
NFR-26	Operational cost estimate to be produced (see Section 3.8).

2.4 Requirement Tensions and Trade-offs

Several requirements directly conflict with each other. These tensions are not problems to avoid. They are the primary drivers of architectural decisions. Each tension below maps to a specific pattern or design choice in Section 3.

Conflict	Tension	Our Response
NFR-01 (99.999% uptime) vs NFR-14 (all tx via CBS)	The Mainframe is a single point of failure on the critical path. If it goes down, we cannot process transactions.	Store and Forward pattern: queue transactions in Kafka during Mainframe outages, then retry automatically on recovery. Circuit Breaker prevents cascade failures.
NFR-02 (zero data loss) vs NFR-26 (minimise cost)	Redundancy costs money. Full replication for everything is expensive.	Selective redundancy: synchronous replication for financial data only. Async storage tiers for non-critical data like report caches.
NFR-06 (ms latency) vs NFR-14 (all tx via CBS)	The Mainframe cannot respond in milliseconds under high concurrent load.	CQRS pattern: reads are served from Redis and PostgreSQL (milliseconds). Only writes go to the Mainframe. CDC keeps the read layer synchronised.
NFR-05 (1M users) vs NFR-14 (all tx via CBS)	1M users would overwhelm the Mainframe and destabilise branch banking operations.	Rate Limiting at BFF level. The digital layer absorbs all read traffic. Only validated writes reach the CBS, throttled to a safe TPS budget.

3. High Level Design

This section describes the architecture from the top down: who uses the system and why (use cases), where things run (deployment), what the pieces are (components), how they are built (patterns and technology choices), and how data flows through them (system flows).

3.0 Use Case Analysis

Before making any technology or deployment decisions, we identified every actor who interacts with the system and every goal they want to accomplish. This ensures that every architectural choice can be traced back to a real need.

3.0.1 Actors

Actor	Type	Description
Existing Customer	Human (primary)	Uses the mobile app or web portal to manage accounts, make transfers, and access digital services.
Third-Party Fintech	External org	Accesses customer data or initiates payments through the Open Banking API under PSD2.
System Administrator	Human (internal)	Monitors system health, responds to alerts, manages infrastructure.
Data Protection Officer	Human (internal)	Reviews and authorises GDPR data deletion requests.
Fraud Analyst	Human (internal)	Reviews offline fraud reports and escalates confirmed cases.
Bank Administrator	Human (internal)	Manages digital user accounts: creation, credential reset, access revocation.
Branch Bank Teller	Human (internal)	Read-only access to customer accounts on behalf of in-branch customers.
AI Service Provider	External system	Processes anonymised customer data and returns financial advice.

3.0.2 Use Cases

ID	Use Case	Actor(s)
UC-01	Login	Existing Customer, Branch Bank Teller
UC-02	View Account and Balance	Existing Customer
UC-03	View Transaction History	Existing Customer
UC-04	Make Fund Transfer	Existing Customer
UC-05	View Fraud Alert Details	Existing Customer
UC-06	View Monthly Report	Existing Customer
UC-07	Request Financial Advice	Existing Customer, AI Service Provider
UC-08	Manage Third-Party Consent	Existing Customer, Third-Party Fintech
UC-09	Request Data Deletion	Existing Customer
UC-10	Access Customer Account Data	Third-Party Fintech
UC-11	Initiate Payment on Behalf of Customer	Third-Party Fintech
UC-12	Monitor System Health and Logs	System Administrator
UC-13	Authorise Data Deletion	Data Protection Officer
UC-14	Review Fraud Detection Reports	Fraud Analyst
UC-15	Provide Financial Analysis	AI Service Provider
UC-16	Manage User Accounts	Bank Administrator
UC-17	View Customer Account on Behalf	Branch Bank Teller

3.0.3 Use Case Diagrams

The use cases are organised across three diagrams by actor group.

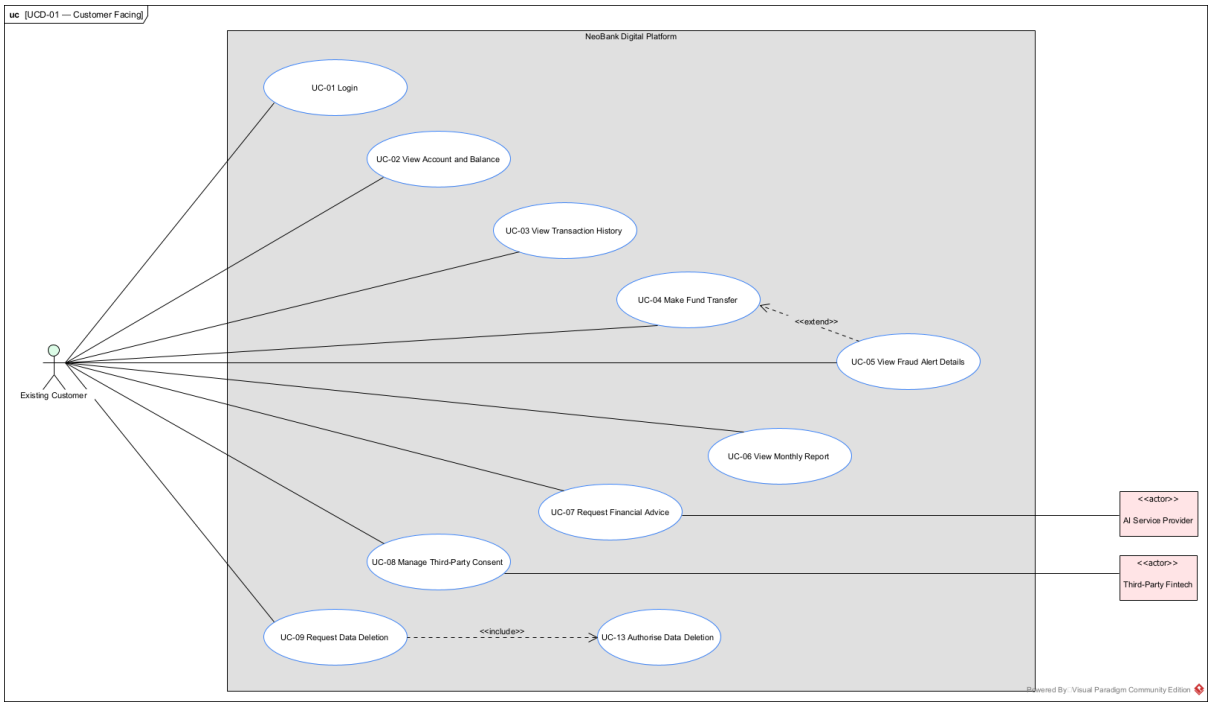

Figure 1: UCD-01: Customer Facing Use Cases

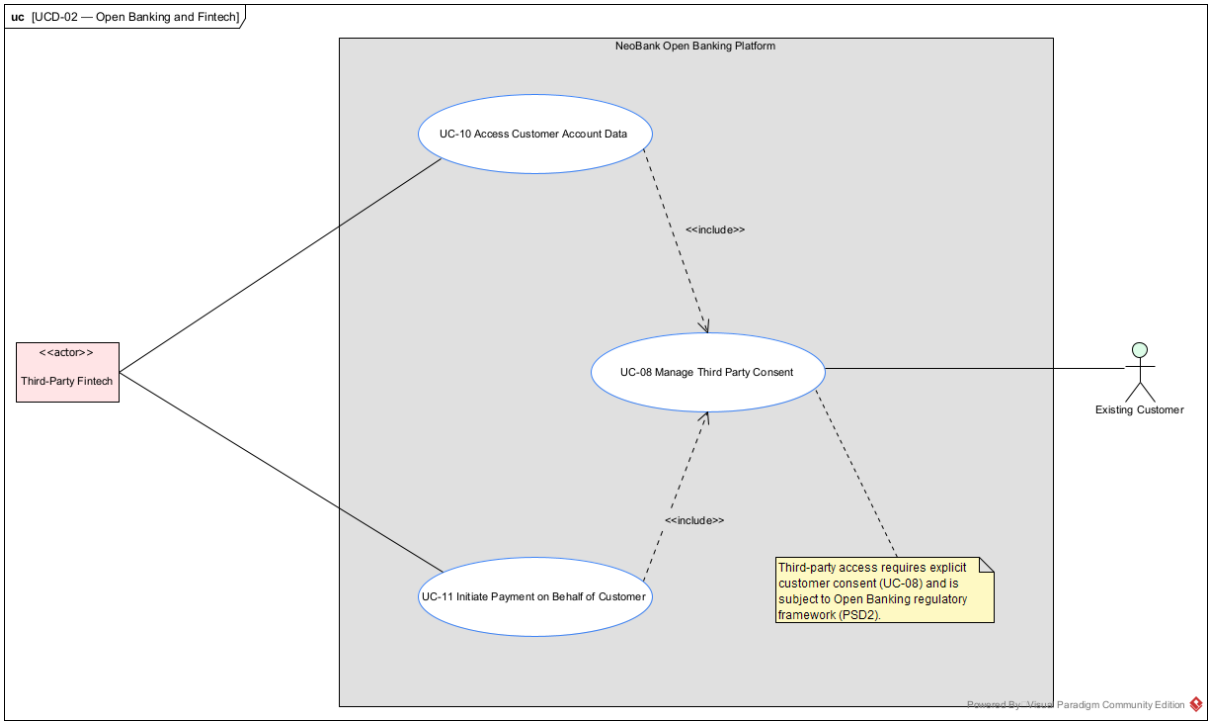

Figure 2: UCD-02: Open Banking and Fintech Use Cases

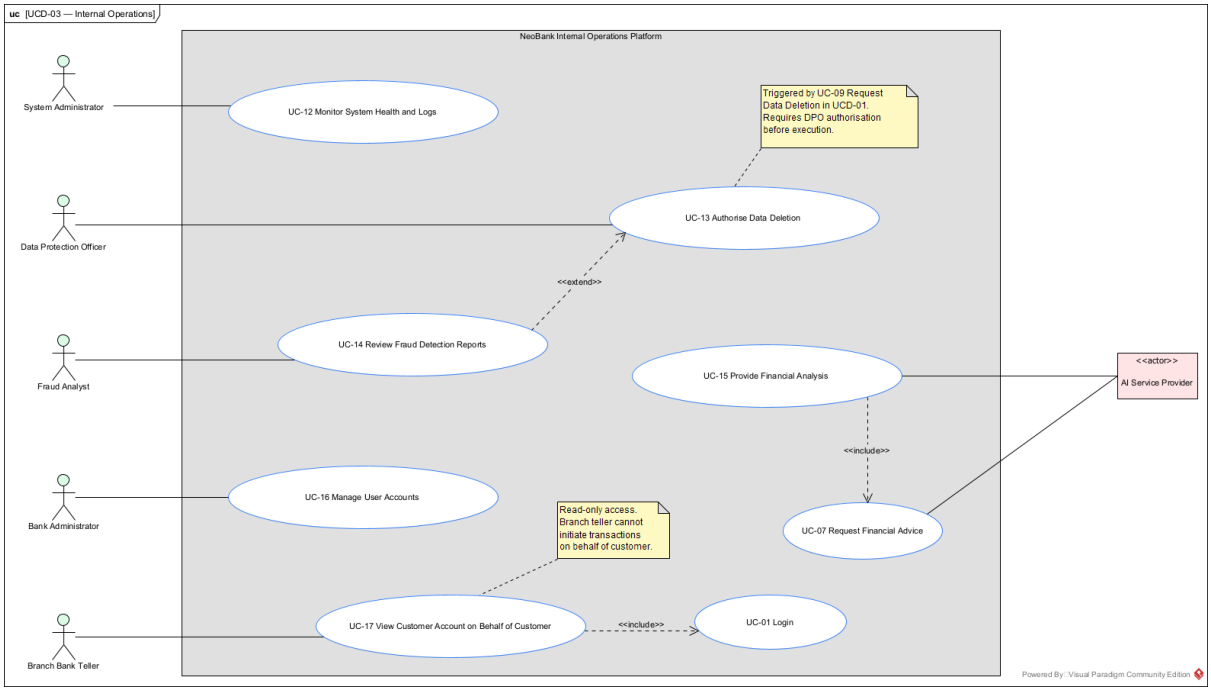

Figure 3: UCD-03: Internal Operations Use Cases

3.1 Deployment and Component Architecture

The system runs across two environments connected by an encrypted VPN tunnel. The split is driven directly by three requirements: NFR-21 (core transactional services on-premises), NFR-22 (API Gateway, Partner Gateway, and AI Agent in the cloud), and NFR-23 (PII stays in the geographic region).

One decision that deserves explanation is the placement of the Data Warehouse, Offline Fraud Detection, and Reporting Service in the cloud. The Data Warehouse uses Amazon Redshift, which is a cloud-only managed service. It cannot run on-premises. Since the Offline Fraud Detection and Reporting Service are batch workers that read terabytes from Redshift, putting them on-premises would mean pulling all that data across the VPN for every batch run. That would be slow, expensive, and unnecessary. Neither of these services is on the critical transaction path, so NFR-21 does not apply to them. PII in Redshift is protected by EU-region deployment, which satisfies NFR-23.

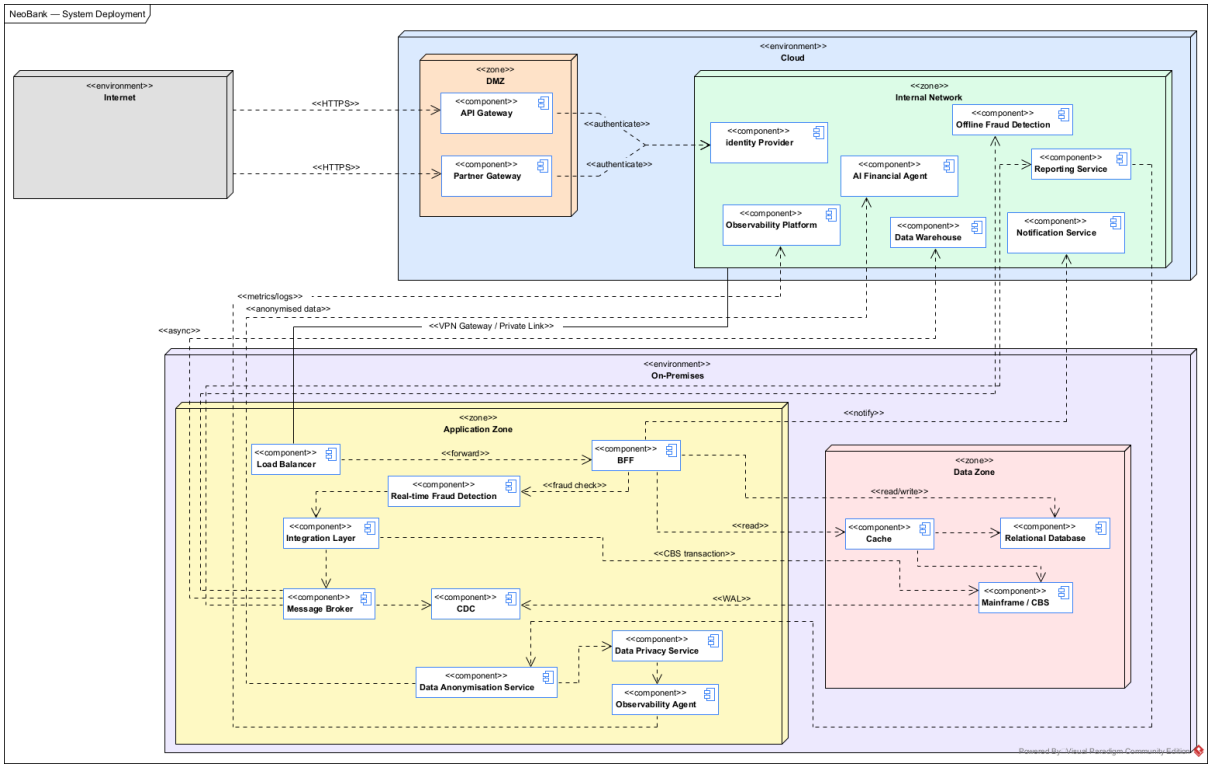

Figure 4: NeoBank System Deployment Diagram

3.1.1 Component Inventory

The table below lists every component, what it does, and where it runs. The environment assignment for each component is derived from the requirements, not from preference.

Component	Responsibility	Env.
API Gateway	Entry point for mobile/web traffic. Authentication, TLS termination, routing.	Cloud
Partner Gateway	Entry point for third-party fintechs. Separate rate limits and security policies.	Cloud
Identity Provider (Keycloak)	Authentication and authorisation for all users and third-party apps.	Cloud (EU)
AI Financial Agent	Personalised financial advice from anonymised data. Cloud-only per FR-05.1.	Cloud
Notification Service	Push notifications such as fraud alerts and confirmations. Uses APNs/FCM.	Cloud
Observability Platform	Centralised logs, metrics, and traces from both environments.	Cloud
Data Warehouse (Redshift)	Analytical store for historical transaction data. Cloud-only managed service.	Cloud (EU)
Offline Fraud Detection	Batch analysis of historical data for complex fraud patterns (FR-02.2).	Cloud
Reporting Service	Monthly income/expense reports per user (FR-03). Reads from Redshift.	Cloud
VPN Gateway / Private Link	Encrypted tunnel between cloud and on-premises. No public internet transit.	Boundary
Load Balancer (ALB)	Distributes traffic across BFF instances for horizontal scaling.	On-prem
BFF	Core transactional service. Orchestrates all client requests.	On-prem
Real-time Fraud Detection	Synchronous analysis of every transfer before CBS submission (FR-02.1).	On-prem
Integration Layer	Bridges Mainframe and modern layer. Forwards writes, propagates confirmations.	On-prem
Message Broker (Kafka)	Decouples async workloads. Safety buffer during CBS outages.	On-prem
Cache (Redis)	In-memory store for balances and profiles. Millisecond reads.	On-prem
Relational DB (PostgreSQL)	Operational store for financial and customer data synced from Mainframe.	On-prem
Data Anonymisation Service	Strips PII before data crosses to cloud. GDPR boundary enforcement.	On-prem
Data Privacy Service	Orchestrates Right to be Forgotten deletions across all data stores.	On-prem
Debezium (CDC)	Reads Mainframe DB2 WAL passively and publishes change events to Kafka.	On-prem
Observability Agent	Collects local metrics and logs, then ships them to the cloud Observability Platform.	On-prem

Mainframe / CBS	System of Record. Every transaction must be confirmed here to be final.	Legacy

3.1.2 Component Diagrams

COMP-01 shows all system components and their interfaces. This is the primary structural reference.

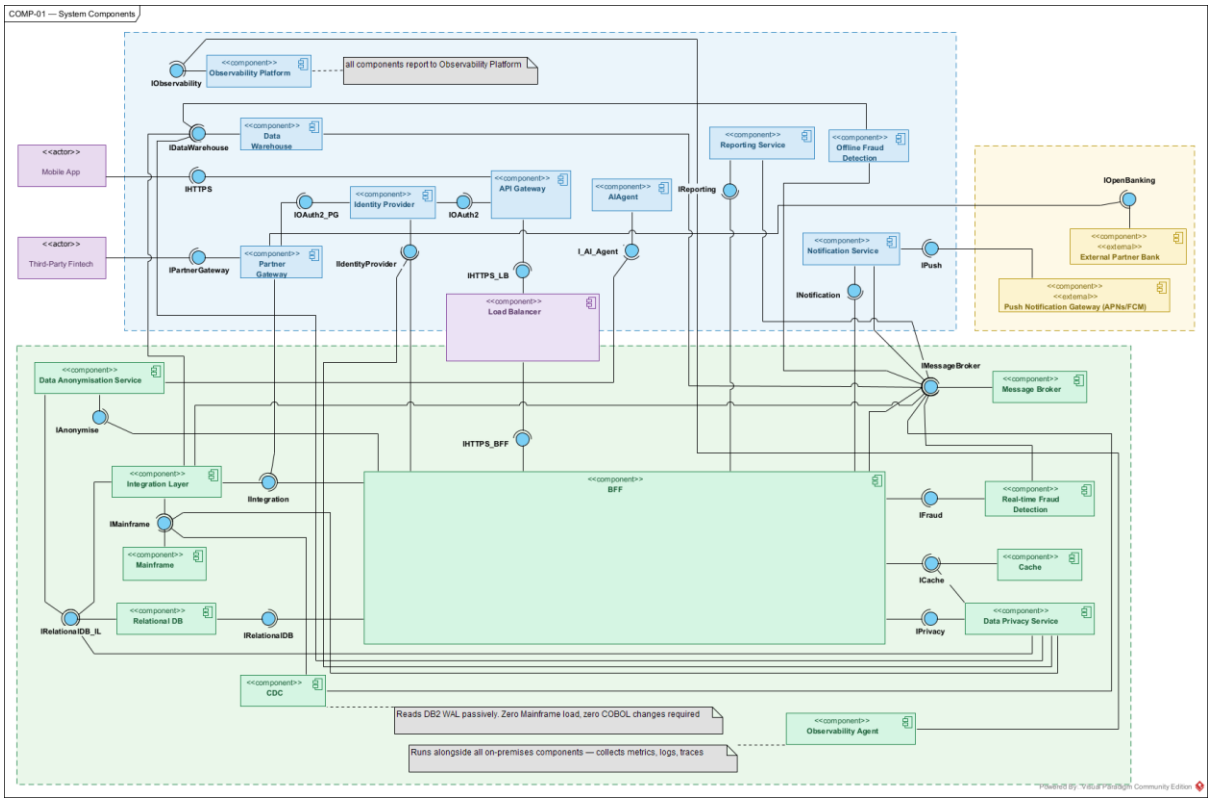

Figure 5: COMP-01: System Components

COMP-02 shows the internal structure of the BFF. It implements four patterns: Rate Limiting, Circuit Breaker, CQRS (with a CommandHandler and QueryHandler), and Cache-Aside (CacheManager).

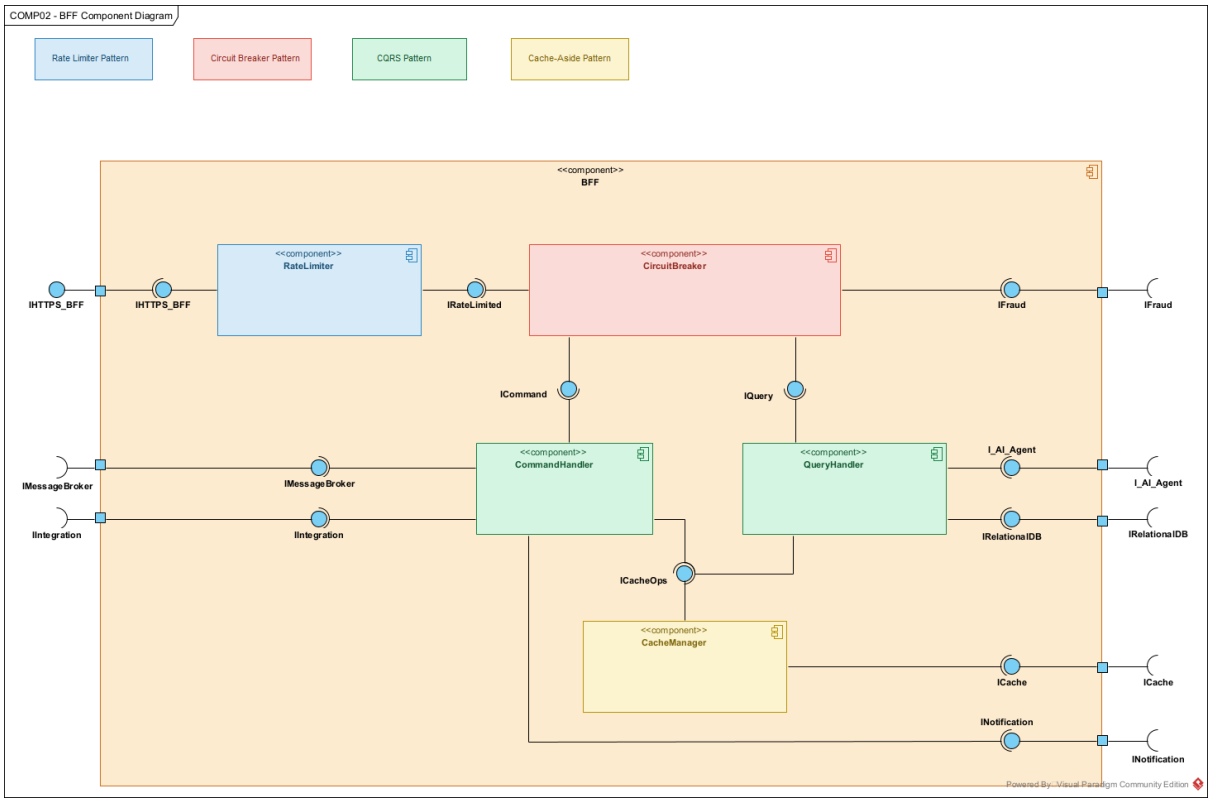

Figure 6: COMP-02: BFF Component Diagram

COMP-03 shows the internal structure of the Integration Layer. It implements four patterns: Store and Forward (TransactionRouter), Outbox (OutboxWriter and OutboxPublisher), Inbox (InboxChecker), and Retry with Idempotency (RetryManager).

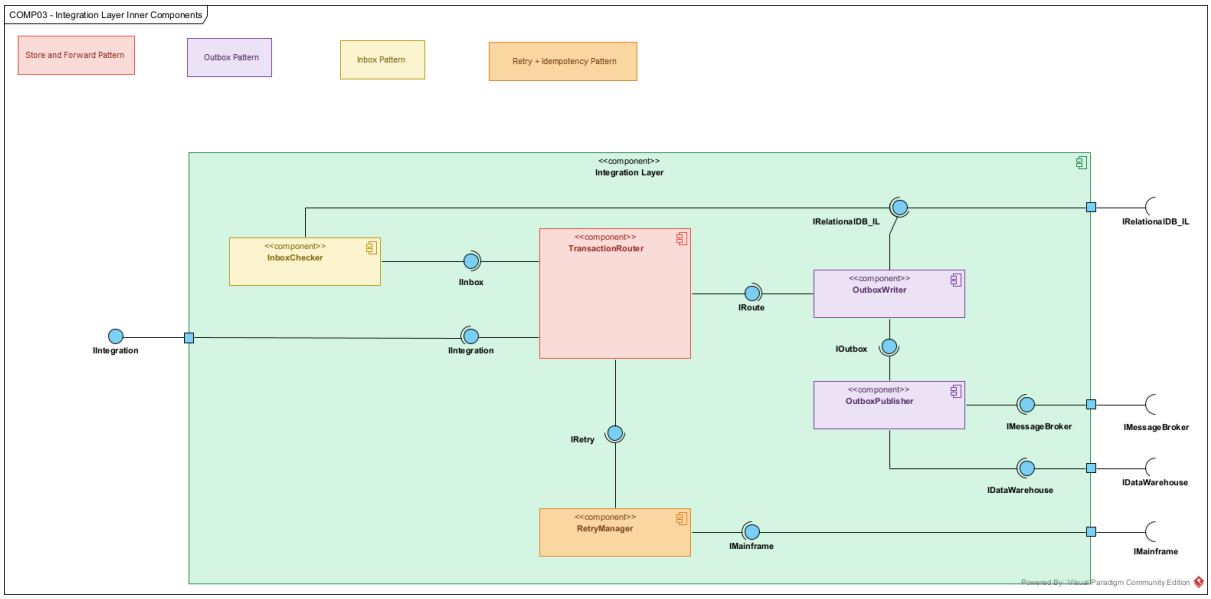

Figure 7: COMP-03: Integration Layer Inner Components

3.1.3 Domain Model

The domain model captures the core business entities. Customer and Account are abstract classes with concrete subtypes. Transaction is abstract with three subtypes: Transfer, Deposit, and Withdrawal. FraudAlert is linked to Transaction (0..1) and can trigger a Notification. ExternalAccount is a standalone entity and not a subtype of Account, because external accounts do not support the same operations. Making it a subtype would violate the Liskov Substitution Principle.

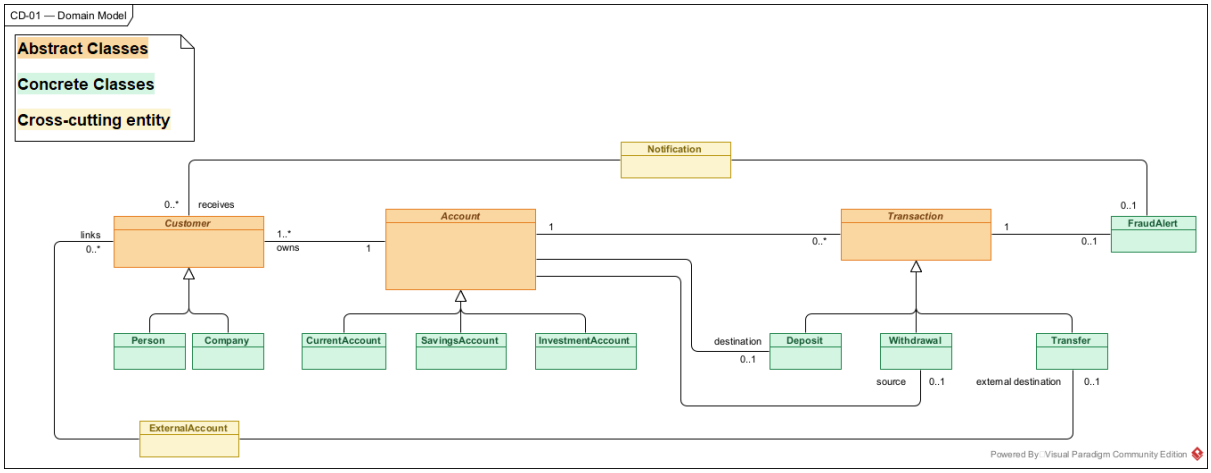

Figure 8: CD-01: Domain Model

3.2 Technology Decisions

Each technology choice below was evaluated against at least one alternative. We document both the decision and the reasoning, so that future team members understand not just what we chose but also why we rejected the alternatives.

Component	Technology	Key Justification	Rejected Alternative
Message Broker	Apache Kafka	Message persistence and independent consumer offsets guarantee NFR-02 (zero data loss). Consumers can replay events.	RabbitMQ: deletes messages after acknowledgement, no replay capability.
Cache	Redis	TTL support and disk persistence. Millisecond latency for NFR-06. Survives restarts without data loss.	Memcached: no TTL per entry, no persistence, all data lost on restart.
Relational DB	PostgreSQL	Strict ACID compliance for financial data. Complex query support. Mature AWS RDS integration.	MySQL: weaker ACID guarantees under concurrent write workloads.
Data Warehouse	Amazon Redshift	Cloud-managed, EU region deployment for GDPR. Native AWS ecosystem. Columnar storage optimised for analytical queries.	Google BigQuery: different cloud platform, adds cross-cloud GDPR complexity.
CDC Tool	Debezium	Reads DB2 transaction log passively. Zero Mainframe load, zero COBOL changes. Open source.	Custom polling: adds load to the Mainframe and violates Design Principle 6.
Identity Provider	Keycloak	Free at any scale. Credentials stay under bank control. No vendor lock-in.	AWS Cognito: per-user cost at 1M scale and credentials stored in AWS infrastructure.
Container Orchestration	Kubernetes	Auto-restart, rolling deployments, horizontal auto-scaling. Industry standard for enterprise workloads.	Docker Swarm: insufficient enterprise-grade features for a banking environment.
CI/CD	GitHub Actions	Cloud-agnostic. Self-hosted runners support hybrid deployment. Reduces AWS lock-in.	AWS CodePipeline: designed for pure AWS environments, not hybrid.

3.3 Design Principles and Architecture Patterns

The principles below are not preferences. They are mandatory constraints derived from the requirement tensions in Section 2.4. The patterns that follow are the specific mechanisms that implement these principles.

3.3.1 Design Principles

Principle	Rationale
Mainframe is SoR: nothing is final until CBS confirms.	NFR-14. Any parallel ledger would risk inconsistency and regulatory exposure.
Decouple reads from writes: reads never touch the Mainframe.	Resolves NFR-06 vs NFR-14. Mainframe reserved for writes only.
Protect the Mainframe from traffic: throttle everything.	Resolves NFR-05 vs NFR-14. Uncontrolled traffic would break branch banking.
Design for Mainframe unavailability: queue, never drop.	Resolves NFR-01 vs NFR-14. Must degrade gracefully and auto-recover.
GDPR by design: no raw PII leaves on-premises.	NFR-24. Anonymise before any data crosses the cloud boundary.
Minimise Mainframe-side changes.	5 COBOL devs are scarce. Every CBS change is a delivery risk.

3.3.2 Structural Patterns

Client-Server

The mobile app is a thin client. All logic runs server-side. The client never talks directly to any backend component. It only communicates with the API Gateway over HTTPS. This keeps the Mainframe completely invisible from the internet (NFR-11).

Backend for Frontend (BFF)

One dedicated service aggregates backend calls and returns a tailored response to the client. This reduces round trips, which helps achieve millisecond latency (NFR-06), and centralises cross-cutting concerns like rate limiting and circuit breaking. The downside is that the BFF becomes a single orchestration point. We mitigate this with Kubernetes horizontal scaling and the Load Balancer.

API Gateway and Microservices

The API Gateway handles authentication at the system boundary (NFR-08). Behind it, the backend is decomposed into independently deployable services (BFF, Fraud Detection, Integration Layer, and others). Each service owns its own data and scales independently, which supports NFR-05 and NFR-15.

3.3.3 Data Patterns

CQRS (Command Query Responsibility Segregation)

This is arguably the most important pattern in the entire architecture. Reads go to Redis and PostgreSQL. Writes go through the Integration Layer to the Mainframe. The two paths are completely separate. This resolves the core tension: we get millisecond reads (NFR-06) without violating the rule that all transactions must go through the CBS (NFR-14). The BFF implements this with a QueryHandler for the read path and a CommandHandler for the write path. See COMP-02 for details.

The trade-off is that read data may be slightly stale. CDC keeps the lag under one second for most operations, and NFR-13 explicitly permits up to 24 hours for external transactions. Transactions initiated through the app are updated synchronously.

Cache-Aside

For every read, the BFF checks Redis first. If the data is in the cache, it returns immediately. If not, it queries PostgreSQL, populates the cache, and then responds. This is the mechanism that delivers the millisecond latency promise (NFR-06). See SD-01 for the complete flow.

Materialised View

Redis stores pre-computed account balances. These are updated by the Integration Layer after every confirmed transaction. We never compute balances from transaction history on the fly because that would be far too slow for the most frequent operation in the system.

Change Data Capture (CDC)

Debezium watches the Mainframe’s DB2 transaction log and publishes every change to Kafka. The modern data layer stays synchronised without touching the COBOL code. We considered custom polling, but that would have added load to the Mainframe and violated Design Principle 6. Debezium reads the WAL passively, meaning the Mainframe does not even know it is there. See SD-07 for the full flow.

Event Sourcing (Partial)

Kafka topics are immutable, append-only logs. Within the retention window, events are never modified or deleted. This gives us a complete audit trail (NFR-24) and the ability to replay events for debugging. We call it 'partial' because the Mainframe remains the system of record. Kafka supports audit and replay, but it does not replace the CBS as the authoritative state.

3.3.4 Resilience Patterns

Circuit Breaker

The BFF wraps calls to the Integration Layer and Fraud Detection in a circuit breaker. If either service fails repeatedly, the circuit opens and the BFF returns a degraded response immediately instead of waiting for timeouts. This prevents a failing fraud engine from bringing down balance queries. The two should be independent (NFR-01). See COMP-02.

Store and Forward

When the Mainframe is down, we do not drop the transaction and we do not show the customer an error. Instead, the Integration Layer queues it in Kafka and retries automatically when the CBS recovers. The customer sees '202 Accepted, processing' and gets a notification when it completes. This is how we maintain uptime even when the single most critical component is unavailable (NFR-01). See SM-01, QUEUED state.

Retry with Idempotency

The RetryManager in the Integration Layer retries failed Mainframe submissions with exponential backoff. Every transaction carries a unique idempotency key, so if a retry arrives after the original has already succeeded (due to a network timeout), the CBS safely ignores the duplicate. This guarantees NFR-02 and NFR-14 together. See COMP-03.

Saga Pattern (Orchestration)

A fund transfer touches multiple systems: BFF, Integration Layer, Mainframe, PostgreSQL, and Redis. If the Mainframe confirms but the database update fails, we are in an inconsistent state. The Saga pattern handles this: the Integration Layer publishes a CompensationEvent to Kafka, and the compensation handler reverses the partial changes. The customer is notified immediately. This is the most complex pattern in the system and the one that will require the most thorough testing.

3.3.5 Messaging Patterns

Outbox Pattern

Consider this failure mode: the Integration Layer commits a transaction to PostgreSQL, then tries to publish an event to Kafka, but crashes between the two operations. The transaction is committed but the event is lost. The Outbox pattern prevents this. The event is written to an outbox table in the same database transaction as the business data. A separate OutboxPublisher reads the table and publishes to Kafka. If the service crashes, the event is still in the outbox and gets published on recovery. See COMP-03.

Inbox Pattern

The inverse problem: Kafka might deliver the same message twice (at-least-once delivery). The InboxChecker looks up the message ID in an inbox table before processing. If it is already there, the duplicate is acknowledged and ignored. Combined with the Outbox pattern, this gives us effectively exactly-once processing. See COMP-03.

Publish-Subscribe (Broker Topology)

Kafka is the central broker. Producers publish events to topics. Multiple independent consumer groups subscribe and process at their own pace. We can add new consumers (for example, a future analytics service) without modifying any producer. This is how the system stays extensible (NFR-05).

3.3.6 Deployment Patterns

Lambda Architecture

The system has two parallel data processing paths. The speed layer runs on-premises: Kafka delivers events in real-time to the Real-time Fraud Detection, and the Integration Layer updates Redis and PostgreSQL immediately. The batch layer runs in the cloud: Redshift accumulates historical data, and the Offline Fraud Detection and Reporting Service process it in scheduled batches. Both feed the serving layer (Redis, PostgreSQL, Redshift) which the BFF queries.

This is not just a data pattern. It maps cleanly to our deployment split. The speed layer is on-premises because it is on the critical transaction path (NFR-21). The batch layer is in the cloud because it needs elastic compute for terabytes of data and it is not time-critical.

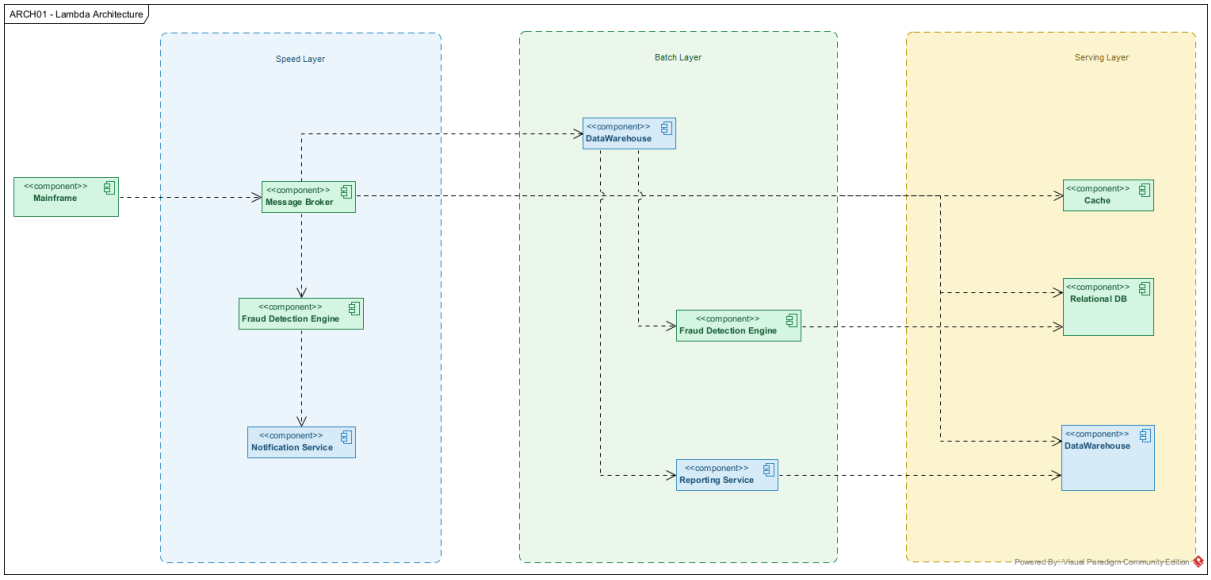

Figure 9: ARCH-01: Lambda Architecture

Throttling and Rate Limiting

The BFF's RateLimiter caps requests per client per time window. This is the last line of defence before traffic reaches the Mainframe. Even if the API Gateway's own throttling is misconfigured, the BFF ensures the CBS never receives more than its safe TPS budget. See COMP-02.

3.4 Security Architecture

Security is not a separate layer. It is woven into every component and every flow. This section describes the security mechanisms, mapped to the STRIDE threat model categories.

STRIDE Category	Threat	Mitigation in NeoBank
Spoofing	An attacker impersonates a legitimate user or service.	OAuth 2.0 with MFA via Keycloak (SD-03). JWT tokens for all API calls. Partner Gateway requires separate OAuth tokens for fintechs. No service-to-service call is unauthenticated.
Tampering	An attacker modifies data in transit or at rest.	TLS encryption for all data in transit (NFR-10). Encrypted storage at rest for PostgreSQL, Redis, and Redshift. Kafka message signing. Idempotency keys prevent replay attacks.
Repudiation	A user or service denies having performed an action.	Kafka event log provides an immutable audit trail (partial Event Sourcing). All API calls are logged with user ID, timestamp, and action. Distributed tracing across all components (NFR-18).
Information Disclosure	PII is exposed to unauthorised parties.	Data Anonymisation Service strips PII before cloud transmission. VPN Gateway encrypts all cross-environment traffic. No Mainframe data accessible from the internet (NFR-11). GDPR Right to be Forgotten via Data Privacy Service (SD-05).
Denial of Service	An attacker overwhelms the system.	Rate Limiting at API Gateway and BFF levels. Circuit Breaker prevents cascade failures. Kubernetes auto-scaling absorbs legitimate traffic spikes. Kafka absorbs async workload bursts.
Elevation of Privilege	An attacker gains access beyond their authorisation.	Role-based access control via Keycloak (NFR-09). Fintech access is scoped to explicit customer consent (UC-08). Branch Bank Tellers have read-only access. DPO authorisation required for data deletion (UC-13).

The deployment diagram enforces a DMZ pattern: the API Gateway and Partner Gateway sit in the cloud DMZ zone, isolated from both the public internet and the internal network. Authentication happens at this boundary, so unauthenticated requests are rejected before they reach the VPN (NFR-08).

3.5 High Level System Flows

The following flows document the critical user journeys and system processes.

3.5.1 Transaction Lifecycle

The state machine shows every state a transaction can pass through, from initiation to completion, failure, or compensation. This is the single most important behavioural diagram in the HLD.

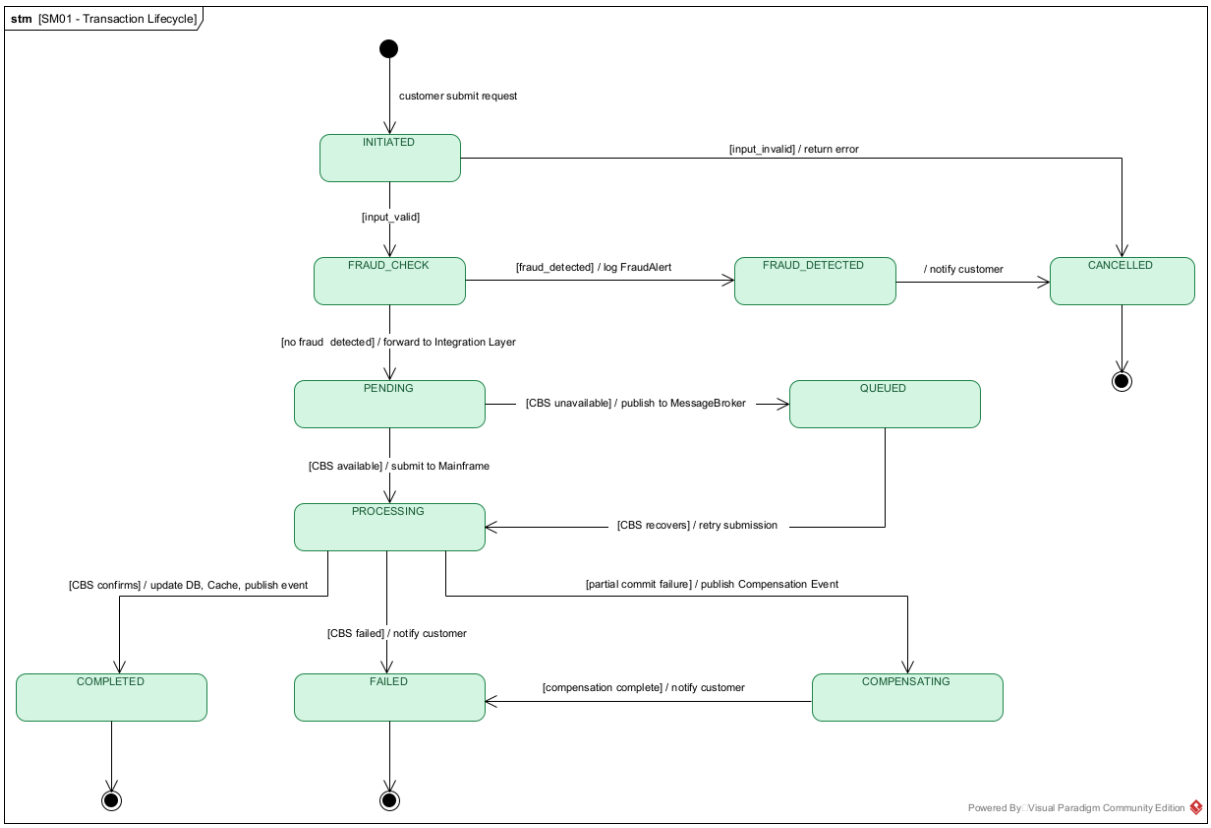

Figure 10: SM-01: Transaction Lifecycle

3.5.2 Account Balance Query (Synchronous Read)

The most frequent operation. The BFF checks Redis using the Cache-Aside pattern. On a hit, it returns in sub-millisecond time. On a miss, it falls back to PostgreSQL and populates the cache. The Mainframe is never involved in a read.

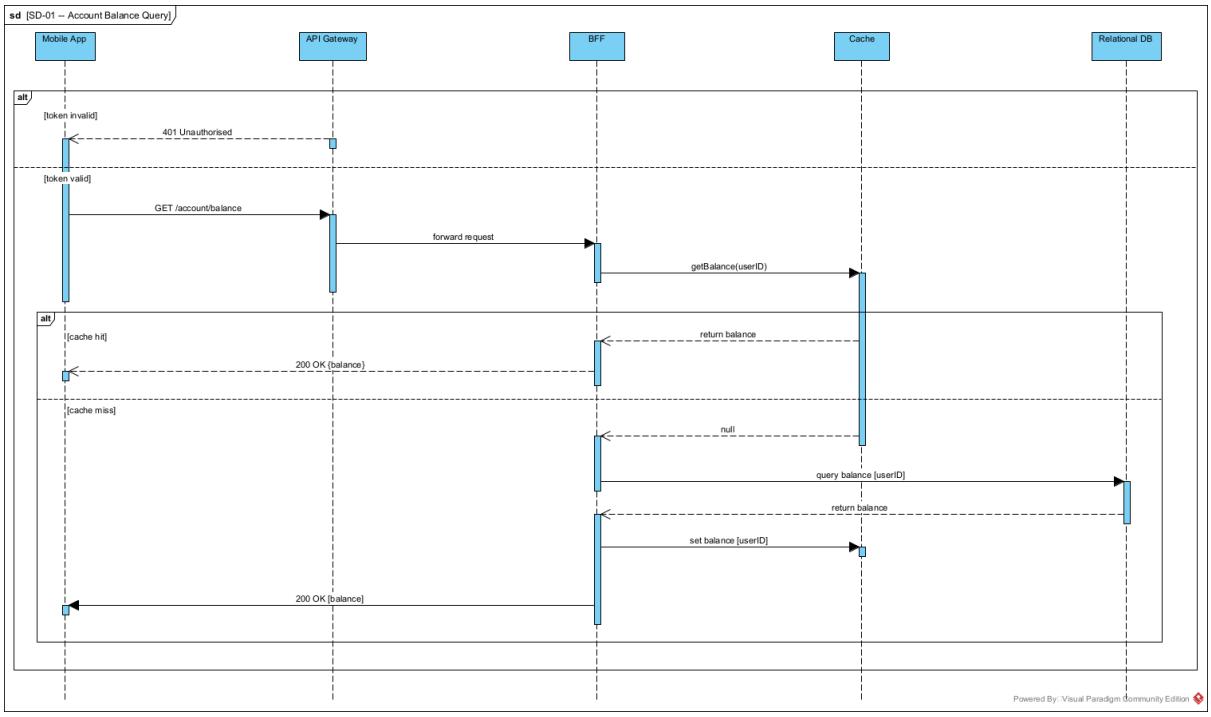

Figure 11: SD-01: Account Balance Query

3.5.3 Fund Transfer with Fraud Detection (Synchronous Write)

The most complex flow. It involves balance validation, real-time fraud detection, Mainframe submission, synchronous data store updates, and asynchronous event publication. The activity diagram shows the complete decision logic across all swim lanes. The sequence diagram shows the exact message sequence.

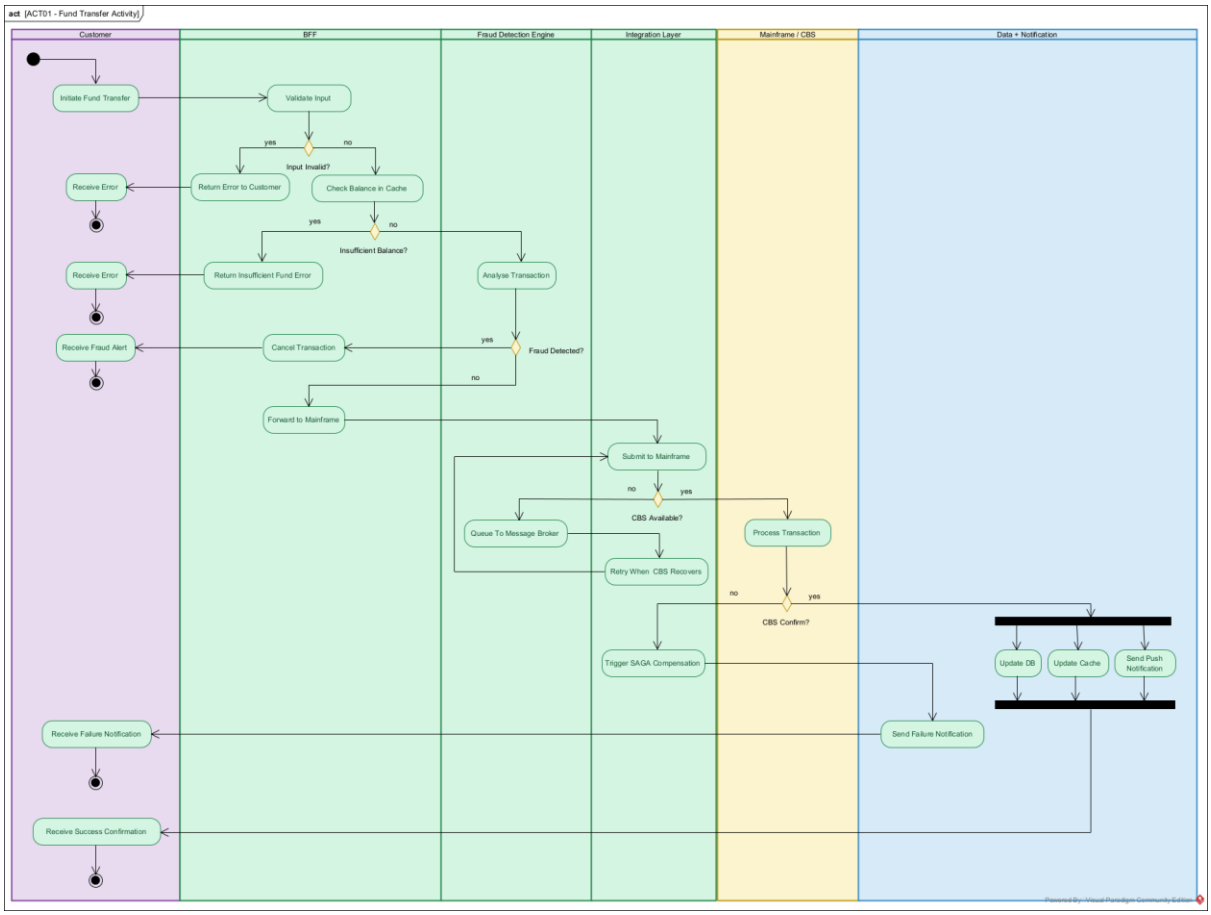

Figure 12: ACT-01: Fund Transfer Activity Diagram

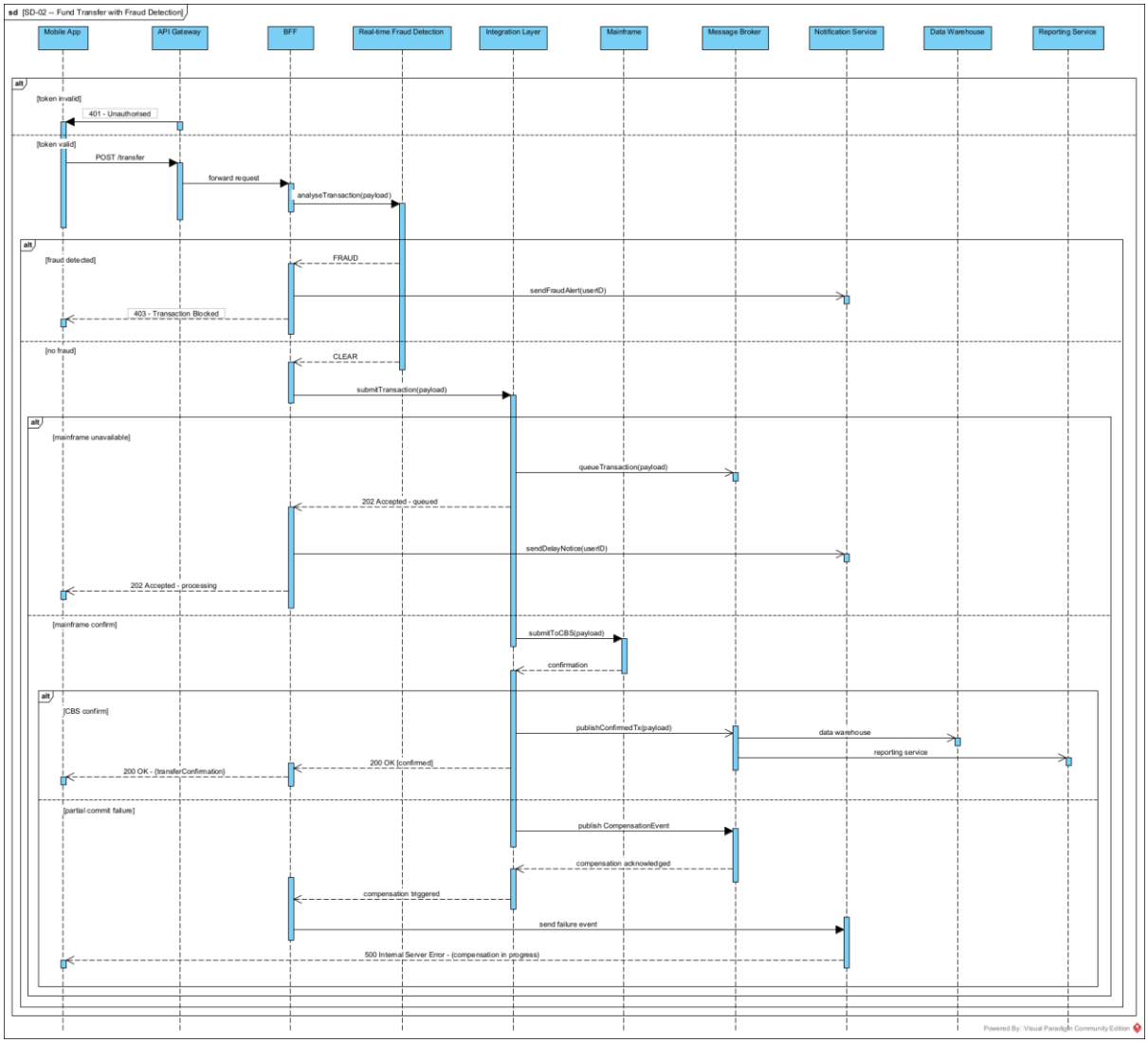

Figure 13: SD-02: Fund Transfer with Fraud Detection

3.5.4 Login and Token Issuance

This flow implements OAuth 2.0 Authorization Code flow with MFA. The user provides credentials, receives an OTP, submits it, and exchanges the authorisation code for a JWT access token stored securely on the device.

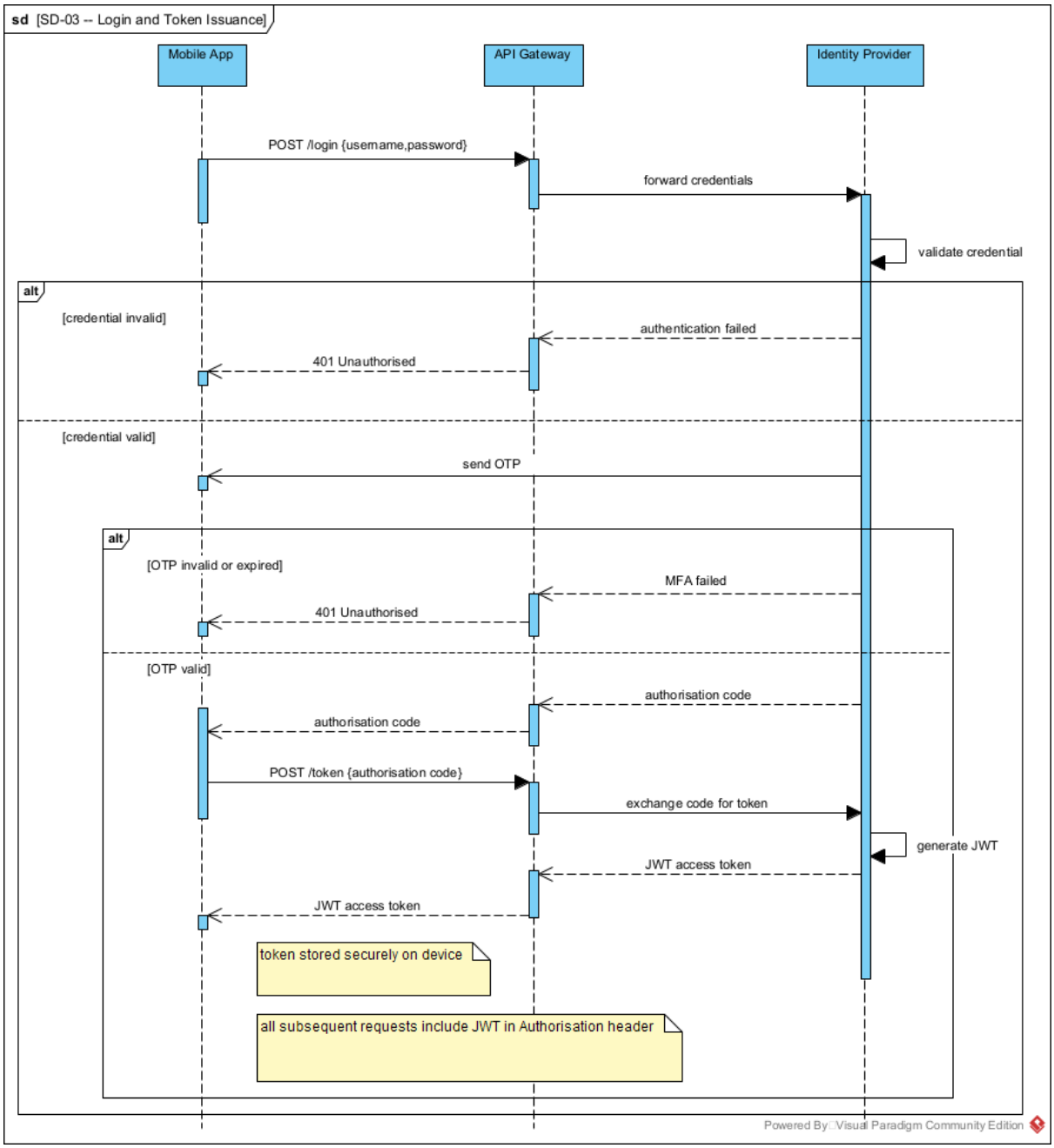

Figure 14: SD-03: Login and Token Issuance

3.5.5 AI Financial Advice

This flow shows the GDPR boundary in action. Customer data is fetched from PostgreSQL, anonymised on-premises by the Data Anonymisation Service, and only then sent to the cloud AI Agent. The response is delivered asynchronously via push notification.

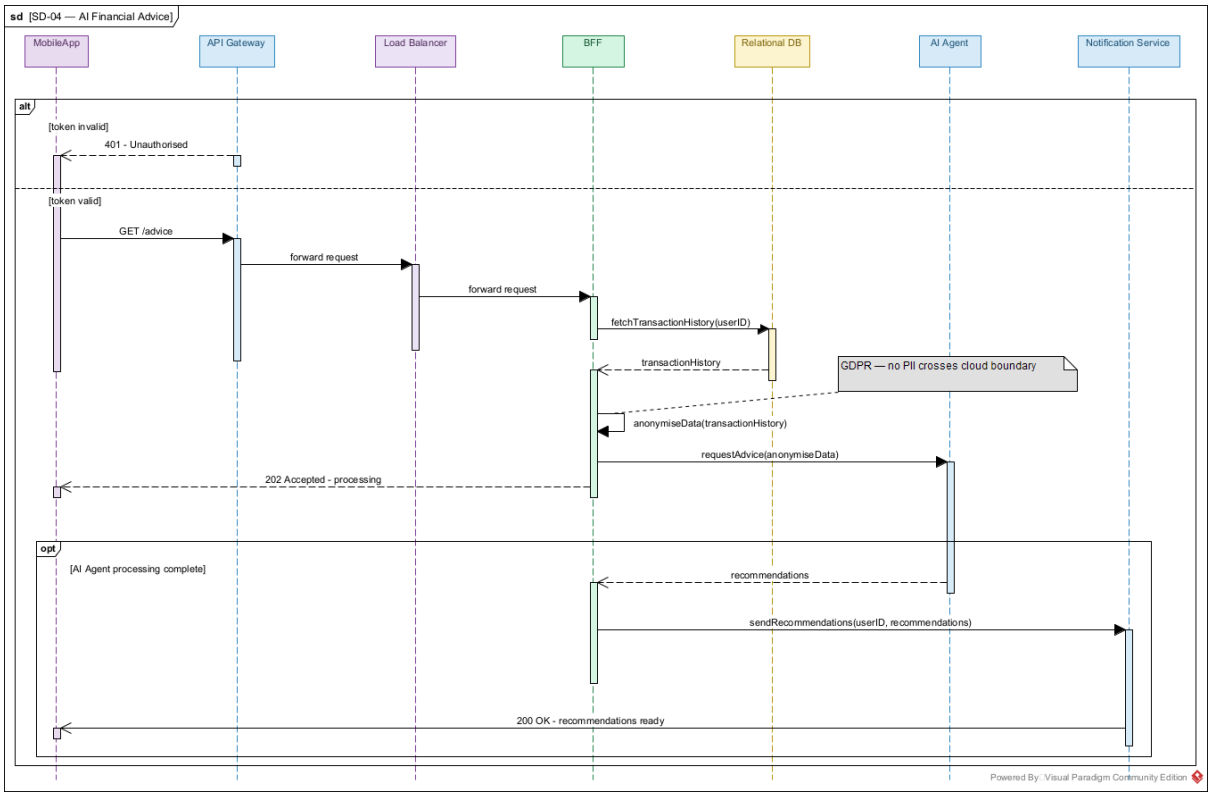

Figure 15: SD-04: AI Financial Advice

3.5.6 GDPR Data Deletion

This flow orchestrates the Right to be Forgotten across every data store. The Data Privacy Service coordinates parallel deletions from Redis, PostgreSQL, Redshift, and Keycloak. Mainframe deletion is best-effort, subject to the COBOL team's capacity. DPO authorisation is required before execution begins.

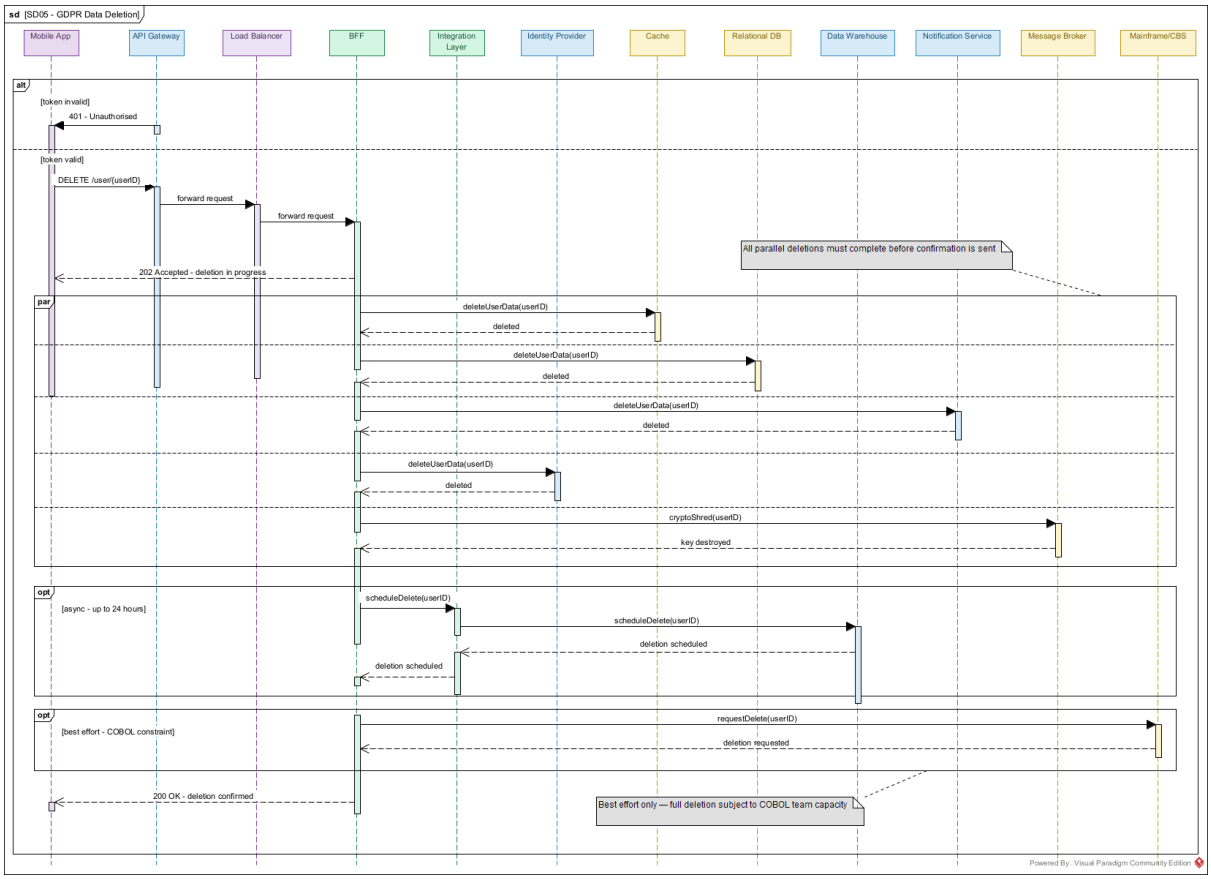

Figure 16: SD-05: GDPR Data Deletion

3.5.7 Open Banking Payment

Third-party fintech payments follow the same pipeline as customer-initiated transfers. FR-02.1 requires fraud detection on every fund transfer, with no exceptions. The Partner Gateway authenticates via OAuth, then routes through the BFF and Real-time Fraud Detection before reaching the Integration Layer.

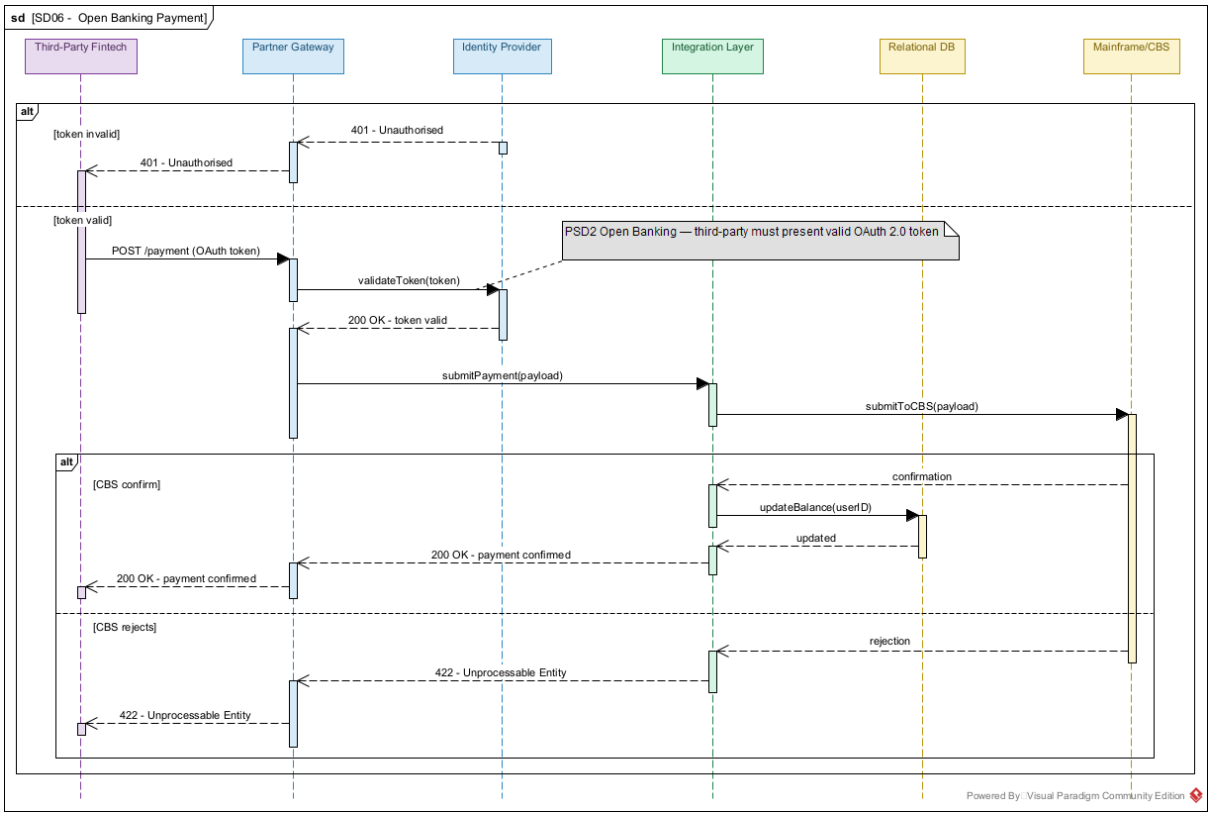

Figure 17: SD-06: Open Banking Payment

3.5.8 CDC Data Pipeline

This flow shows how we keep the modern layer in sync with the Mainframe without touching COBOL. Debezium reads the DB2 WAL passively and publishes to Kafka. Consumer services update PostgreSQL, Redis, and Redshift independently.

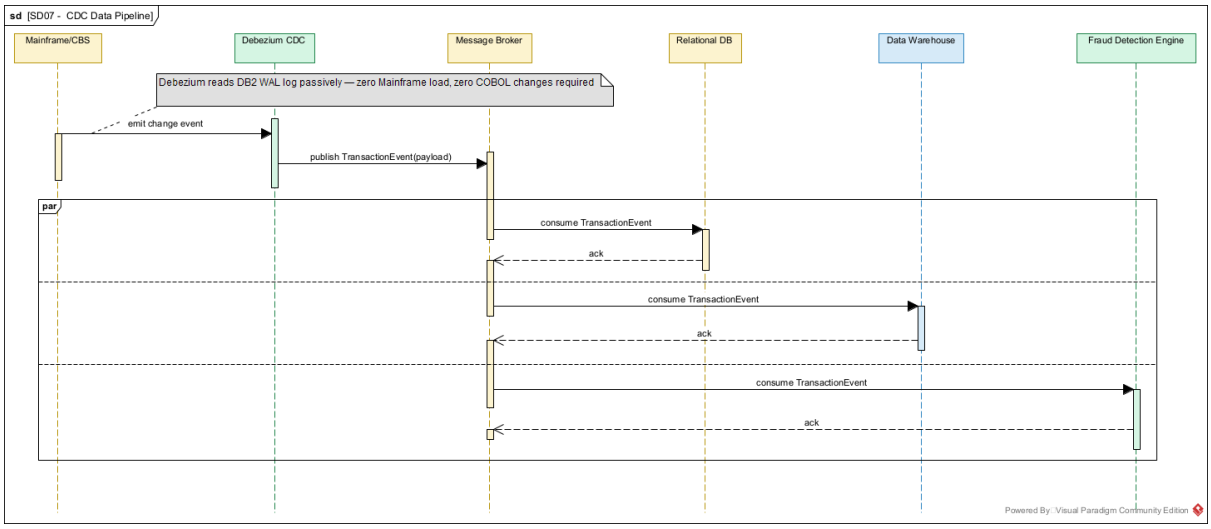

Figure 18: SD-07: CDC Data Pipeline

3.6 Message Schemas

Key messages exchanged between components. Detailed API contracts will be defined in the Low Level Design.

Message	From / To	Protocol	Key Fields
POST /login	App to API GW	HTTPS	username, password
POST /verify-otp	App to API GW	HTTPS	otp, sessionID
GET /account/balance	App to BFF	HTTPS	userID (JWT)
POST /transfer	App to BFF	HTTPS	sourceAcct, destAcct, amount, currency, idempotencyKey
POST /payment	Fintech to BFF	HTTPS+OAuth	sourceAcct, destAcct, amount, consentToken
analyseTransaction	BFF to RT Fraud	gRPC	txID, sourceAcct, amount, riskProfile
submitTransaction	BFF to Integ. Layer	gRPC	txID, payload, idempotencyKey
submitToCBS	Integ. Layer to Mainframe	MQ/CICS	txID, CBS-formatted payload
TransactionEvent	Debezium to Kafka	Kafka (Avro)	txID, type, amount, acctID, timestamp
ConfirmedTxEvent	Integ. Layer to Kafka	Kafka (Avro)	txID, status, amount, accounts
FraudAlertEvent	Offline Fraud to Kafka	Kafka (Avro)	alertID, txIDs[], riskScore, pattern
CompensationEvent	Integ. Layer to Kafka	Kafka (Avro)	txID, reason, compensationAction

3.7 Upgradability

We designed for zero-downtime deployments from day one (NFR-15).

Rolling Deployment

All on-premises services run as Kubernetes pods with multiple replicas. New versions deploy one pod at a time. The old pod is only terminated after the new one passes its health check. At no point are all instances of a service unavailable.

Blue-Green for Database Migrations

Backward-incompatible schema changes use two phases. Phase 1 deploys the new schema alongside the old one (dual-write). Phase 2 retires the old schema after all services have been verified (NFR-16).

Kafka Topic Versioning

When a message schema changes in an incompatible way, we create a new topic version (for example, transactions.v2). The Avro schema registry enforces compatibility at publish time. Consumers are migrated before the old topic is deprecated.

Feature Flags

New features deploy behind flags. This allows gradual rollout to a subset of users and instant rollback without redeployment.

3.8 Sizing

These estimates are based on 100K users at MVP, scaling to 1M in 3 years. All numbers must be validated during the infrastructure spike in the first month.

Traffic

Metric	MVP (100K)	Year 3 (1M)
DAU (20% of total)	20,000	200,000
Balance queries per day	100,000	1,000,000
Transfers per day	10,000	100,000
Peak read TPS	approx. 10	approx. 100
Peak write TPS	approx. 1	approx. 10

Compute

Component	MVP	Year 3	Per Instance
BFF	3 pods	10 pods	2 vCPU, 4 GB RAM
RT Fraud Detection	2 pods	6 pods	4 vCPU, 8 GB RAM
Integration Layer	2 pods	6 pods	2 vCPU, 4 GB RAM
Kafka	3 brokers	5 brokers	4 vCPU, 16 GB, 500 GB SSD
Redis	1 + 1 replica	1 + 2 replicas	2 vCPU, 8 GB RAM
PostgreSQL	1 + 1 standby	1 + 2 standbys	4 vCPU, 32 GB, 500 GB SSD

Storage

Store	Year 1	Year 3
PostgreSQL	approx. 50 GB	approx. 500 GB
Redis	approx. 2 GB	approx. 20 GB
Kafka (7-day retention)	approx. 20 GB	approx. 200 GB
Redshift	approx. 100 GB	approx. 2 TB

4. Time Estimation

The team of 60 developers is split into parallel workstreams. The Mainframe Integration workstream is on the critical path. Everything else can start independently, but if the Mainframe bridge fails, the whole project stalls.

Workstream	Team	Duration	Deliverables
Mainframe Integration	5 COBOL + 5 Java	12 months	CDC pipeline, Mainframe adapter, Integration Layer, CBS interface
On-Premises Services	15 devs	10 months	BFF, RT Fraud Detection, Cache layer, PostgreSQL schema, Kafka setup
Cloud Services	10 devs	8 months	API GW, IdP, Notifications, Redshift, Offline Fraud, Reporting, AI Agent
Mobile and Web	10 devs	8 months	iOS/Android app, web portal, UX design, client-side security
DevOps and Infra	8 devs	12 months	Kubernetes, CI/CD, monitoring, VPN, security hardening
QA	12 devs	10 months	Integration tests, load tests, security tests, UAT

A two-week spike in the first sprint will validate Assumption A-01 (can Debezium read the DB2 WAL on z/OS?). If it fails, we activate the batch extraction fallback immediately. We cannot afford to discover this problem at month six.

The AI Financial Agent is scoped as optional for MVP. If GDPR compliance for cloud AI cannot be resolved within 6 months, the feature is deferred. The core banking platform does not depend on it.

5. Limitations

These are realities we must work within. They are not problems we can solve.

Limitation	Impact
Legacy Integration Complexity	The COBOL and z/OS environment was built for batch processing, not real-time mobile interactions. Building the bridge is the hardest part of this project.
COBOL Team Bottleneck	Only 5 COBOL developers for the entire project. Any Mainframe-side work is a scarce resource on the critical path.
Legal Service Locality	Some services must run on-premises by law. This is permanent and cannot be changed by architectural choice.
Fragmented Legacy Data	Transactions (DB2) and customer information (ADABAS) are in two separate systems that were never designed to be queried together.
GDPR vs AI Data Richness	Anonymisation reduces data quality. The AI Agent may give less useful advice because of it. This is a fundamental trade-off with no easy solution.
Aggressive Timeline	Hybrid infrastructure, Mainframe integration, fraud detection, AI, Open Banking, and a mobile app in 12 months is extremely ambitious.

6. Risks and Mitigations

Risk	Impact	Mitigation
Mainframe CDC proves infeasible (A-01 fails)	Critical: the whole integration strategy collapses.	Two-week spike at project start. Batch extraction fallback defined in advance.
Digital traffic overwhelms the Mainframe	Critical: could destabilise branch banking.	Caching and async absorption for reads. Rate limiting for writes. Max TPS budget enforced at BFF level.
12-month deadline missed	High: competitors gain time.	Freeze scope early. Use managed services. Defer AI Agent if necessary.
COBOL team becomes a bottleneck	High: delays cascade everywhere.	Single scoped workstream. No context switching. Minimum Mainframe changes by design.
GDPR compliance for AI Agent unresolvable	Medium: AI is a differentiator but not launch-critical.	Treat as optional MVP. Run GDPR compliance in parallel but do not gate launch on it.

7. Open Issues

ID	Description	Impact	Next Steps
OI-01	CDC feasibility on z/OS not validated	Critical	Two-week spike, Sprint 1
OI-02	IdP deployment may require on-prem per banking regulations	Medium	Legal review, month 1
OI-03	RTO not yet defined (NFR-03)	Medium	DR analysis before Sprint 3
OI-04	Maintenance windows not agreed (NFR-17)	Low	Stakeholder workshop
OI-05	AI Agent GDPR path unconfirmed	Medium	DPO review in parallel
OI-06	Partner Gateway technology not formalised	Low	Cloud workstream kickoff
OI-07	Detailed cost estimate pending (NFR-26)	Low	After infra spike